

Cryptographic Cipher Analyzer

Caesar & Vigenere Cryptanalysis Tool - CLI internship project

Version 1.0.0 | Generated 15 September 2026

EDUCATIONAL USE ONLY

This project demonstrates classical cryptography and cryptanalysis. Caesar and Vigenere ciphers are historically important but are NOT secure modern encryption algorithms. Do not use this project to protect passwords, credentials, financial information, personal data or confidential communications.

1. What this project is

The Cryptographic Cipher Analyzer is a command-line tool that implements the Caesar and Vigenere ciphers from scratch and then attacks them with classic cryptanalysis: letter frequency analysis, chi-square scoring, Index of Coincidence, Kasiski examination and per-column key recovery. Every algorithm is written in pure Python using the standard library only - no cryptography packages, no web interface and no hidden magic.

- Language: Python 3.11 or newer (developed and verified on 3.13).
- Dependencies: none at runtime - argparse, collections, math, statistics, re, pathlib, json, datetime, unittest.
- Platforms: Windows, Linux, Kali Linux, Ubuntu (identical commands everywhere).
- Two ways to work: an interactive menu, or scriptable single-command mode.

2. Installation and first run

No installation step is required. Fetch the project and run it:

```
# 1. Open a terminal in the repository folder
cd crypto_cipher_analyzer

# 2. Check the Python version (3.11+ required)
python --version

# 3. Start the interactive menu
python main.py

# 4. Or run a single command (no menu)
python main.py caesar-encrypt --text "HELLO WORLD" --shift 3

# 5. On Linux/macOS use python3 if 'python' is not mapped
python3 main.py --help

# 6. Optional: create a virtual environment first
python -m venv .venv
# Windows: .venv\Scripts\activate
# Linux:   source .venv/bin/activate
```

Console notes

Windows: if 'python' opens the Microsoft Store, use the 'py' launcher instead: py main.py

The tool auto-detects colour support and falls back to plain text. Add the global flag

--no-color (before the command) or set the NO_COLOR environment variable to force it.

3. Interactive menu reference (python main.py)

Running the tool with no arguments prints the banner and the menu below. Invalid selections are reported and the menu is shown again, so the program never crashes on bad input.

```
=====
      CRYPTOGRAPHIC CIPHER ANALYZER
      Caesar & Vigenere Cryptanalysis Tool
=====

Purpose: Educational cybersecurity and cryptography analysis tool

MAIN MENU
-----
[1] Caesar Cipher (encrypt / decrypt)
[2] Vigenere Cipher (encrypt / decrypt)
[3] Frequency Analysis
[4] Break Caesar Cipher (automatic)
[5] Analyze Vigenere Cipher (cryptanalysis)
[6] File Encryption / Decryption
[7] Security Analysis
[8] Help
[9] Exit
```

Menu	What it does	Asks you for
1	Caesar encrypt or decrypt, with an explanation of why the cipher is weak.	Encrypt/decrypt, the text (typed or from a file) and the shift.
2	Vigenere encrypt or decrypt with a letters-only keyword.	Encrypt/decrypt, the text and the keyword.
3	Letter frequency table, comparison with standard English and the Index of Coincidence.	The text to analyse.
4	Tries all 26 shifts, scores each candidate and shows the best plaintext.	The ciphertext only - no shift needed.
5	Combined cryptanalysis: IC, Kasiski, key-length candidates, per-column key letters and candidate keys.	The ciphertext, then the key length to use.
6	Encrypt or decrypt a text file (Caesar or Vigenere) without touching the original.	Input file, output file, then the shift or keyword.
7	Security ratings, strengths, weaknesses and known attacks for both ciphers.	Nothing - it is a report.
8	Command reference plus short explanations of every technique.	Nothing.
9	Exit the program.	Nothing.

Whenever the menu asks for text, the prefix 'INPUT SOURCE' lets you type the text (1) or read it from a file (2). Enter 'b' to go back. Every analysis can be saved to a report at the end.

4. Command-line reference

Every menu entry also exists as a sub-command, which makes the tool easy to script and easy to demonstrate. Global options must be written before the command name.

```
python main.py --version          # print the version and exit
python main.py --help             # list all commands
python main.py --no-color <COMMAND> ... # disable ANSI colours for this run
```

Command	Purpose
interactive	Start the menu (also the default when no command is given).
caesar-encrypt	Encrypt text with the Caesar cipher.
caesar-decrypt	Decrypt Caesar ciphertext.
vigenere-encrypt	Encrypt text with the Vigenere cipher.
vigenere-decrypt	Decrypt Vigenere ciphertext.
frequency	Single-letter frequency analysis plus English comparison and IC.
crack-caesar	Break Caesar ciphertext automatically (all 26 shifts are ranked).
analyze-vigenere	Full Vigenere cryptanalysis and key recovery.
ic	Index of Coincidence, including the average IC per key length.
kasiski	Kasiski examination: repeated sequences, distances, factor ranking.
security	Security / strength assessment of both classical ciphers.
file-caesar	Encrypt or decrypt a file with the Caesar cipher.
file-vigenere	Encrypt or decrypt a file with the Vigenere cipher.

4.1 Option reference

Option	Applies to	Meaning
--text "..."	all text commands	The input text. Quote it so the shell keeps the spaces.
--input FILE	all text commands	Read the input text from a file instead of --text.
--shift N	caesar-*, file-caesar	Caesar shift; negative values and values above 26 are normalised modulo 26.
--key WORD	vigenere-*, file-vigenere	Keyword containing letters only (for example LEMON).
--report	every analysis command	Save the result as reports/analysis_YYYYMMDD_HHMMSS.txt without asking.
--output FILE	file-caesar, file-vigenere	Destination file; defaults to <name>_out.<ext>.
--decrypt	file-caesar, file-vigenere	Decrypt instead of encrypt.
--overwrite	file-caesar, file-vigenere	Allow replacing an existing output file.
--top N	crack-caesar, analyze-vigenere	How many candidates to display (default 5).
--max-key-length N	analyze-vigenere (20), ic (12), kasiski (20)	Largest key length to consider.
--key-length N	analyze-vigenere	Skip key-length estimation and recover the key for this length.
--min-length / --max-length	kasiski	Shortest / longest repeated sequence to search for (defaults 3 and 5).
--cipher {caesar,vigenere}	security	Show only one cipher's assessment.

Exit codes (useful in scripts)

0 = success 1 = handled error (invalid input, missing file, ...)

2 = command-line usage error 130 = cancelled with Ctrl+C

Normal users never see a Python traceback: errors are printed as friendly messages.

5. Verified examples with real output

The commands below were executed against the shipped code; the output shown is the actual output (colour stripped).

5.1 Encrypt and decrypt with Caesar

```
$ python main.py caesar-encrypt --text "HELLO WORLD" --shift 3

CAESAR ENCRYPTION
-----
Shift used      : 3
Input length   : 11
Output length  : 11

OUTPUT
-----
KH00R ZRU0G

$ python main.py caesar-decrypt --text "KH00R ZRU0G" --shift 3

OUTPUT
-----
HELLO WORLD
```

5.2 Encrypt and decrypt with Vigenere

```
$ python main.py vigenere-encrypt --text "ATTACKATDAWN" --key LEMON

VIGENERE ENCRYPTION
-----
Key           : LEMON
Key length    : 5
Input length  : 12
Output length : 12

OUTPUT
-----
LXF0PVEFRNHR

$ python main.py vigenere-decrypt --text "LXF0PVEFRNHR" --key LEMON
-> ATTACKATDAWN
```

5.3 Break Caesar without knowing the shift

```
$ python main.py crack-caesar --text "KH00R ZRU0G"
```

CAESAR CRACKING

```
-----  
Ciphertext length : 11  
Candidates evaluated : 26  
Candidates shown : 5  
Ciphertext: KH00R ZRU0G  
Possible solutions (ranked by English-likeness)  
Rank Shift Score Candidate plaintext  
-----  
1 3 40.81 HELLO WORLD  
2 14 26.04 WTAAD LDGAS  
3 25 19.37 LIPPS ASVPH
```

BEST CANDIDATE

```
-----  
Estimated shift : 3  
Score : 40.81  
HELLO WORLD
```

5.4 Frequency analysis, IC, Kasiski and Vigenere analysis

```
$ python main.py frequency --text "KH00R ZRU0G"  
$ python main.py frequency --input sample_data/caesar_sample.txt --report  
$ python main.py ic --text "..." --max-key-length 12  
$ python main.py kasiski --input sample_data/vigenere_sample.txt  
$ python main.py analyze-vigenere --input sample_data/vigenere_sample.txt --key-length 5  
$ python main.py security --cipher caesar  
  
# The shipped Vigenere sample really is LEMON-encrypted:  
$ python main.py analyze-vigenere --input sample_data/vigenere_sample.txt --max-key-length 8  
-> candidate keys include LEMON, and decrypting with it yields readable English
```

6. Working with files

File mode never modifies your original file unless you explicitly ask for it. Without `--output` the result is written next to the input with an `_out` suffix.

```
# Encrypt a file (creates notes_out.txt, notes.txt is untouched)  
python main.py file-caesar --input notes.txt --shift 3  
  
# Decrypt it again into a different file  
python main.py file-caesar --input notes_out.txt --output restored.txt --shift 3 --decrypt  
  
# Vigenere on a file  
python main.py file-vigenere --input notes.txt --output notes_enc.txt --key LEMON  
python main.py file-vigenere --input notes_enc.txt --output notes_dec.txt --key LEMON --decrypt  
  
# Replace an existing output file on purpose  
python main.py file-caesar --input notes.txt --output notes_out.txt --shift 3 --overwrite
```

- Handled gracefully: file not found, a directory instead of a file, permission errors, empty files, an output equal to the input, and an output that already exists.
- The interactive file menu asks before overwriting an existing file.

7. Report generation

Any analysis can be saved. Interactively the tool asks 'Save this analysis as a report? [y/n]'; on the command line add `--report`. Files are written to `reports/` as `analysis_YYYYMMDD_HHMMSS.txt` and contain the date, cipher, operation, input statistics, the frequency table, the cryptanalysis evidence, candidate solutions, the

security assessment and the educational note.

```
# Save a report without any prompt
python main.py crack-caesar --input sample_data/caesar_sample.txt --report

# Then inspect it
ls reports/
cat reports/analysis_20260915_184500.txt
```

8. Running the test suite

```
cd crypto_cipher_analyzer
python -m unittest discover tests          # 163 tests, all passing
python -m unittest discover tests -v       # verbose: one line per test
python -m unittest tests.test_cli -v      # a single module
python main.py --version
```

Test module	Covers
tests/test_caesar.py	Caesar encryption/decryption, shift 0 and 26, negative and wrapping shifts, punctuation, spaces, newlines, non-ASCII, invalid shifts.
tests/test_vigenere.py	Vigenere encryption/decryption, key case-insensitivity, spaces/punctuation/digits, keystream generation, invalid keys.
tests/test_frequency.py	Letter counts, percentages summing to 100, non-letters ignored, most/least common letters, chi-square, digram and correlation scoring.
tests/test_cracker.py	Automatic Caesar cracking, ranking quality, short-text caveats.
tests/test_analysis.py	Index of Coincidence values, interpretations, Kasiski factors and distances, Vigenere key-length and key recovery, security profiles.
tests/test_files.py	File read/write, default output naming, overwrite protection, missing and empty files.
tests/test_cli.py	End-to-end CLI: every command, argparse errors, report writing, interactive flows and clean exit codes.

9. Project structure

```

crypto_cipher_analyzer/
|-- main.py                # CLI entry point: menus, arguments, presentation
|-- README.md              # MISSING - see section 10
|-- requirements.txt        # MISSING - the tool needs no dependencies
|-- LICENSE                 # MISSING
|-- ciphers/
|   |-- __init__.py
|   |-- caesar.py           # encrypt / decrypt (shift, modulo 26)
|   +-- vigenere.py         # encrypt / decrypt (keystream, offsets)
|-- analysis/
|   |-- __init__.py
|   |-- frequency.py        # counts, percentages, chi-square, digrams
|   |-- caesar_cracker.py    # 26-shift search ranked by English-likeness
|   |-- vigenere_analyzer.py # IC + Kasiski + per-column key recovery
|   |-- kasiski.py          # repeated sequences, distances, factors
|   |-- ic_analysis.py       # Index of Coincidence and its interpretation
|   +-- security.py         # strength ratings, attacks, disclaimer
|-- utils/
|   |-- __init__.py
|   |-- text_utils.py       # normalisation, alphabet, English frequencies
|   |-- validators.py       # ValidationError and every input check
|   |-- banner.py          # banner, menu, tables, prompts, colours
|   +-- report_generator.py # timestamped text reports
|-- file_operations/
|   |-- __init__.py
|   +-- file_handler.py     # safe read/write, transform_file, overwrite guard
|-- reports/                # generated reports (.gitkeep is MISSING)
|-- tests/                  # 7 modules, 163 tests
|-- sample_data/
|   |-- caesar_sample.txt    # KHOOR ZRUOG + a long Caesar passage
|   +-- vigenere_sample.txt  # LEMON-encrypted passage + LXFOPVEFRNHR
+-- docs/PROJECT_GUIDE.pdf   # this guide

tools/generate_project_guide_pdf.py # regenerates this PDF (needs reportlab)

```

10. Completion status

Assessment of the project against the internship specification. Every functional requirement is implemented and verified; what is still missing is packaging and documentation, not code.

Spec feature	Status	Evidence / note
1. Main CLI banner and menu	DONE	main.py prints the banner, the purpose line and the 9-entry menu; invalid input is rejected without crashing.
2. Caesar cipher	DONE	ciphers/caesar.py with type hints and docstrings; HELLO WORLD -> KHOOR ZRUOG verified.
3. Vigenere cipher	DONE	ciphers/vigenere.py; ATTACKATDAWN + LEMON -> LXFOPVEFRNHR verified.
4. Frequency analysis	DONE	Table of count and percentage, most/least common letters, comparison with English frequencies.
5. Automatic Caesar cracker	DONE	All 26 shifts scored and ranked; KHOOR ZRUOG -> HELLO WORLD, shift 3 (rank 1).
6. Index of Coincidence	DONE	analysis/ic_analysis.py, correct formula, references and interpretation wording.
7. Kasiski examination	DONE	Sequence lengths 3-5, positions, distances, factors and a ranked key-length list.
8. Vigenere analysis	DONE	Combined IC + Kasiski, key-length selection, per-column letters, candidate keys, decryption preview.

Spec feature	Status	Evidence / note
9. Security / strength analysis	DONE	analysis/security.py: VERY WEAK and WEAK / EDUCATIONAL ONLY with attacks and limits.
10. File operations	DONE	Caesar and Vigenere on files, _out default, overwrite protection, every file error handled.
11. Report generation	DONE	reports/analysis_YYYYMMDD_HHMMSS.txt with date, cipher, statistics, analysis, candidates and the disclaimer.
12. Professional CLI helpers	DONE	utils/banner.py: banner, menu, section, success, error, warning, table; colour auto-disabled when unsupported.
13. Command-line arguments	DONE	All specified commands plus ic, kasiski, security and file-*; supports --text, --input, --output, --shift, --key, --report.
14. Error handling	DONE	ValidationError and FileOperationError give friendly messages; exit codes 0/1/2/130; no tracebacks.
15. Testing	DONE	163 unit tests across 7 modules - all passing (0.6 s).
16. Sample data	DONE	caesar_sample.txt and vigenere_sample.txt, both with the short example and a long passage.
17. README.md	MISSING	Must be written: title, objectives, features, installation, usage, CLI commands, examples, concepts, testing, structure, future work, disclaimer.
18. Code quality	DONE	PEP 8 style, type hints, docstrings, modular design, no hard-coded paths, algorithms separate from the UI.
19. Educational explanations	DONE	Short explanations are printed with each operation (Caesar, Vigenere, frequency, IC, Kasiski, cracking).
20. Security disclaimer	DONE	Printed on start-up and embedded in every generated report.

10.1 Outstanding items

Item	Why it matters	Effort
README.md	Required by the specification (feature 17) and needed for a GitHub submission; screenshot placeholders go in docs/screenshots/.	Small
requirements.txt	The specification asks for it. The tool has no third-party dependencies, so it is an empty/comment-only file.	Trivial
LICENSE	Required by the specification for publication.	Trivial
reports/.gitkeep	Keeps the reports/ folder in version control.	Trivial
docs/screenshots/*	Placeholder images referenced by the README.	Small

10.2 Verified evidence

```
python --version           -> Python 3.13.x
python -m unittest discover tests -> Ran 163 tests ... OK
python main.py --help      -> 13 sub-commands listed
python main.py caesar-encrypt --text "HELLO WORLD" --shift 3 -> KH00R ZRU0G
python main.py caesar-decrypt --text "KH00R ZRU0G" --shift 3 -> HELLO WORLD
python main.py vigenere-encrypt --text "ATTACKATDAWN" --key LEMON -> LXF0PVEFRNHR
python main.py crack-caesar --text "KH00R ZRU0G" -> HELLO WORLD (shift 3, rank 1)
python main.py analyze-vigenere --input sample_data/vigenere_sample.txt
                                -> key length 5, candidate key LEMON recovered
```

Overall

Functional specification: 100% complete and verified (all 20 features implemented; 19 of the 20 specification features are fully in place). Remaining work is packaging only: README.md, requirements.txt, LICENSE, reports/.gitkeep and the screenshot placeholders.
Estimated completion: about 95% of the full internship deliverable.

11. The cryptanalysis in one page each

Useful for the viva or interview discussion of the project.

11.1 Frequency analysis

Natural language is very uneven: in English, E, T and A dominate while Z, Q and X are rare. Counting letters (and comparing the observed percentages with standard English values) shows whether the letter distribution has been preserved, flattened or shifted, which points at the cipher family in use.

11.2 Automatic Caesar cracking

The Caesar cipher has only 26 keys, so all of them are tried. Each candidate plaintext is scored with statistics - a chi-square comparison against English letter frequencies plus a digram score - and the candidates are ranked. The result is estimated, not proven: short messages, names and technical terms can rank the wrong shift first.

11.3 Index of Coincidence

$IC = \sum(f_i(f_i-1)) / N(N-1)$, where f_i is the count of a letter and N the number of letters. English prose sits near 0.0667 and random text near 0.0385. For Vigenere ciphertext the IC of a single column is meaningless, but the average IC of the columns peaks when the assumed key length equals the real one. The IC is evidence, never proof.

11.4 Kasiski examination

Repeated fragments in ciphertext reveal that the repeating key realigned with the plaintext, so the distance between the repeats is a multiple of the key length. The common factors of those distances become candidate key lengths. Kasiski returns hypotheses to verify - a divisor of the true length is always among them, which is why the tool blends it with IC evidence rather than trusting it alone.

11.5 Vigenere key recovery

Once a key length is assumed, position 1, 6, 11, ... form one column, position 2, 7, 12, ... another, and so on. Each column is just a Caesar cipher, so frequency analysis recovers its shift independently. Combining the best letters per column produces candidate keys, each of which is scored by decrypting the whole text and measuring how English-like the result is. The output labels every result as estimated or candidate.

12. Security limitations and disclaimer

- Caesar cipher - rating VERY WEAK: 25 effective keys, broken instantly by brute force or a single frequency table; letter patterns and word lengths survive, and one known letter reveals the entire shift.
- Vigenere cipher - rating WEAK / EDUCATIONAL ONLY: a much larger keyspace, but a repeating key leaks its length through Kasiski examination and the Index of Coincidence; once the length is known each column falls to frequency analysis.
- Neither cipher offers diffusion, randomness or authentication, so ciphertext can be modified undetected. Modern communication needs authenticated encryption (for example AES-GCM or ChaCha20-Poly1305) with properly generated random keys.

EDUCATIONAL USE ONLY

These algorithms are implemented for educational purposes and must NOT be used to protect passwords, credentials, financial information, personal data or confidential communications.

Guide generated by tools/generate_project_guide_pdf.py - regenerate it with 'python tools/generate_project_guide_pdf.py' after the project changes.